LibanPost eShop — Security Architecture

Contents

1. System overview 2
2. Trust boundaries 2
3. Threat model (STRIDE) 3
4. Authentication & session 3
5. Authorization model 3
6. Data protection controls 3
7. Payment flow security 4
8. Vendor payout & commission 4
9. Returns & refunds 4
10. Email & outbox 4
11. Rate limiting & abuse controls 4
12. Image storage security 5
13. Logging & monitoring 5
14. Deployment hardening summary 5
15. Known limitations (security-relevant) 5

Threat model, security controls, and data flow for the marketplace platform.

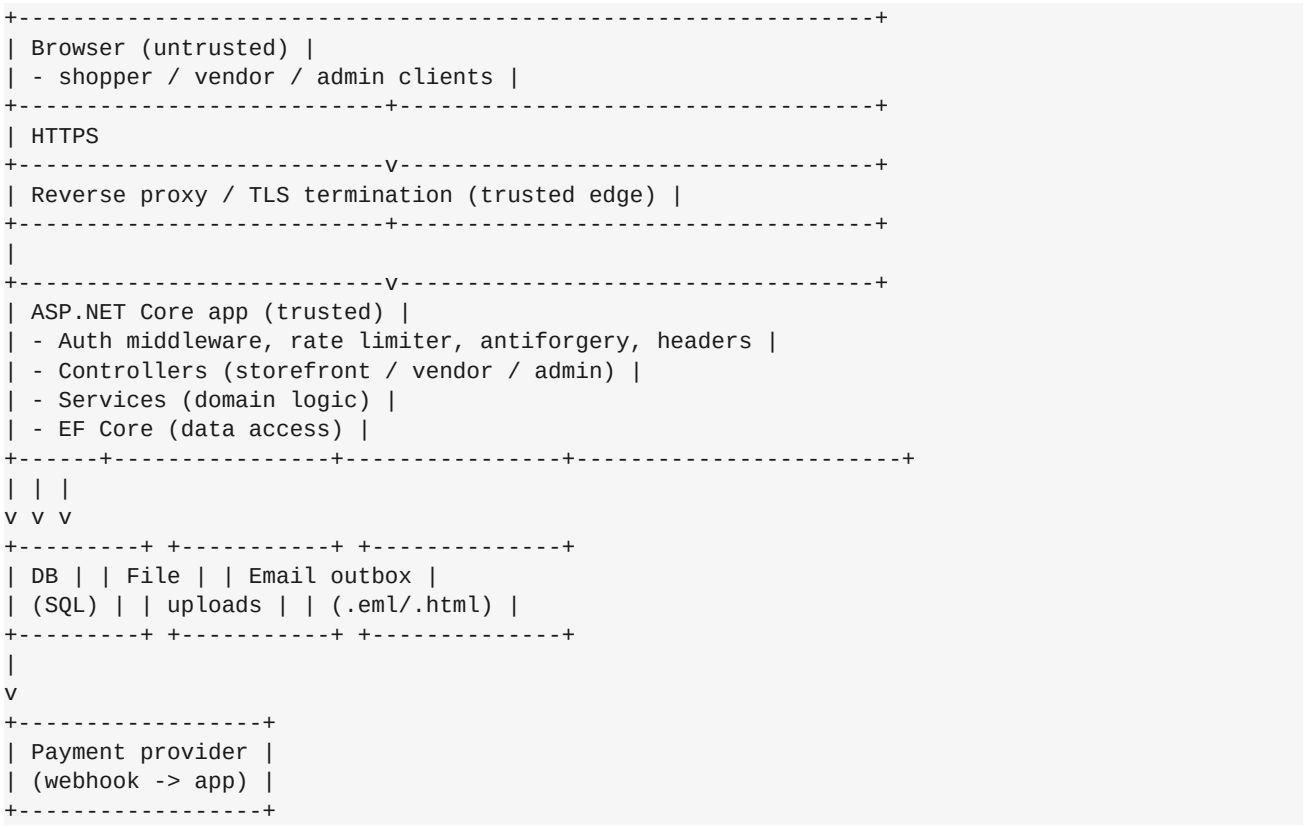
1. System overview

LibanPost eShop is an ASP.NET Core 9 MVC marketplace with three roles:

- **Shoppers** (anonymous or authenticated) browse, cart, checkout, review.
- **Vendors** manage their own shop, products, orders, returns, payouts.
- **Administrators** govern the whole marketplace.

Data stores: SQLite (dev) or SQL Server (prod). File storage for uploads and email outbox.

2. Trust boundaries



Trust boundaries:

- **B1**: Browser ↔ app (all input untrusted).
- **B2**: App ↔ database (trusted, least-privilege creds).
- **B3**: App ↔ file system (uploads, outbox).
- **B4**: Payment provider ↔ app (signed webhook).
- **B5**: Admin ↔ app (privileged, separate authz).

3. Threat model (STRIDE)

Threat	Category	Mitigation
Credential stuffing / brute force	Spoofing	Rate limit `auth` policy; account lockout 5/15min; generic messages.
Session hijacking	Spoofing	HttpOnly cookie; SameSite=Lax; HTTPS/HSTS; sliding expiry.
CSRF	Tampering	Global antiforgery; SameSite cookie.
SQL injection	Tampering	EF Core parameterized queries; no raw SQL from input.
XSS	Tampering	Razor auto-encoding; CSP report-only.
Path traversal in uploads	Tampering	GUID filenames; extension + magic-byte validation; bounded reads.
Malicious SVG upload	Tampering	SVG explicitly rejected (script carrier).
Overselling / race on stock	Tampering	Optimistic concurrency (RowVersion); retry + customer message.
Double charge / double credit	Tampering	Idempotency keys; webhook dedupe; ledger idempotency per order/type/return.
Forged payment webhook	Spoofing	HMAC-SHA256 signature verified before body trusted; 401 on failure.
Replay of webhook	Tampering	Idempotent by (Provider, EventId); `MarkPaidAsync` no-op when already paid.
Privilege escalation (vendor → admin)	Elevation	Policy-based auth; shop membership enforced in base controller.
Cross-tenant data leak (vendor A sees B)	Information disclosure	Workspace scoped by shop; unknown shop → 404.
Sensitive data in logs	Information disclosure	Sink suppressed for its own SQL; retention configurable.
Denial of service	DoS	Rate limiting; bounded queues; export caps; paging clamps.
Account enumeration	Information disclosure	Generic sign-in/reset responses.

4. Authentication & session

- Cookie scheme, 14-day sliding expiry, `HttpOnly`, `SameSite=Lax`.
- Passwords: PBKDF2-HMAC-SHA256, 210,000 iterations, constant-time compare.
- Security stamp rotated on credential change; reset tokens single-use and expiring.
- Lockout: 5 attempts / 15 minutes.

5. Authorization model

- Roles: `Admin`, (vendor via shop membership), (customer via account).
- Policies: `AdminOnly`, `VendorOnly`, `CustomerOnly`.
- Shop-scoped access enforced in `VendorControllerBase` — an order/return/payout for another shop returns 404 (not 403) to avoid existence leaks.
- Admin actions on shops/returns/payouts recorded in the audit trail.

6. Data protection controls

- **Orders snapshot** product name, variant, SKU, unit price → history immutable.
- **Soft delete** products; sold variants deactivated.
- **Optimistic concurrency** on stock (`RowVersion`).
- **Idempotent checkout** (per-form key + unique index).
- **Append-only ledgers**: `StockMovement`, `VendorLedgerEntry`, `AuditLog`, `PaymentEvents`, `CouponUsage`.

- **Money is** `decimal` throughout; SQL Server maps to `decimal(18,2)`.

7. Payment flow security

```
Shopper chooses card
|
v
CreateIntentAsync -> provider URL (simulator in dev)
|
v
Provider posts webhook -> /Payments/Webhook/{provider}
|
v
Read raw body -> verify X-Payment-Signature (HMAC-SHA256)
|
+-- invalid -> record SignatureValid=0, return 401
|
+-- valid -> parse event -> idempotency check (PaymentEvents)
-> MarkPaidAsync (no-op if already paid; never revives cancelled/refunded)
-> credit vendor ledger (when money captured)
```

- Declined payments leave the order `Pending` with a `Failed` attempt; retry allowed.
- Card refunds go against the original charge; COD refunds are staff bookkeeping rows.

8. Vendor payout & commission

- Commission withheld at the gate, goods only (never shipping), rounded half-away-from-zero.
- Credit occurs when money is collected (card capture / COD delivery / counter paid).
- Balance is **derived** (sum of unsettled ledger entries), never stored.
- Payouts are settlements (stamp entries `SettledUtc` + reference); no new earnings written.
- Reversal on cancellation/refund; idempotent per `(order, type, return/post-sale)`.
- Settlement outcomes: Paid / Failed / Reversed; failed/reversed releases covered entries.

9. Returns & refunds

- Lifecycle: Requested → Approved → Received → Refunded (with Rejected / Cancelled exits).
- One open return per order; per-unit scope enforced server-side.
- Refund computed (goods share of discount + proportional tax; shipping only on whole-order return), capped by ledger owed and `GrandTotal − RefundedTotal`.
- Ledger reverses proportionally; goods restocked with return-stamped movement.
- Refund issuance is staff-only; shops decide/receive but never move money.

10. Email & outbox

- Nothing sends on the request path; rows written to `EmailMessages`, drained by background dispatcher.
- Idempotent by `DedupeKey`; retries with backoff up to `Email:MaxAttempts`, then parked Failed.
- Resend returns a failed message to the queue with clean history.
- Password reset uses signed, expiring tokens; resetting rotates the security stamp.

11. Rate limiting & abuse controls

- Fixed-window limiter, partitioned by user or IP; 429 + `Retry-After` on rejection.
- Webhook policy deliberately generous (120/min) to survive provider retries.
- Endpoint defended by signature, not rate limit, for webhooks.

12. Image storage security

- Validation: extension allow-list, size cap, magic-byte verification.
- SVG rejected (script carrier).
- GUID filenames in a store-owned folder; no path traversal.
- Failed save cleans up written files; removal deletes file unless still referenced.

13. Logging & monitoring

- Console + `Log` table; `Error`/`Critical` with full stack trace.
- Logging never throws or blocks (bounded queue, background writer).
- Audit trail: append-only, loose references, best-effort writes.

14. Deployment hardening summary

- HTTPS only; HSTS; security headers; forwarded headers for real client IP.
- Secrets via environment variables / secret manager; fail-fast on missing webhook secret.
- Least-privilege DB credentials; backups with point-in-time recovery.
- Upload folder outside web root; served via configured request path.
- Replace `simulator` payment gateway with a real `IHostedPaymentGateway` before production.
- Configure `Logging:Database:RetentionDays` to bound log growth.

15. Known limitations (security-relevant)

- No real payment provider wired (simulator moves no money).
- Email default transport writes files (no real delivery until SMTP configured).
- No full-text search (LIKE) — potential DoS on huge catalogues.
- Uploaded images on local disk; multi-instance needs shared storage.
- Log table has no request path / trace id / user column.

Architecture as implemented. Where code and document disagree, the code is authoritative.